NAME: OM
REGISTER.NO: 192425275

# CO2-AT1- Analytical Problem Solving


## NAME: AJAY KUMAR REDDY

## REGISTER.NO:192425274


## COURSE: CSA0712 - COMPUTER NETWORKS

## FACULTY: Dr. S. SALINI

---

**Experiment 1: Hamming Code Interactive Sandbox**

**Aim**

To generate a Hamming Code for the given binary data and verify the error detection and correction capability by introducing a single-bit error.

**Objective**

The objective of this experiment is to understand the working of Hamming Code and observe how it detects and identifies a single-bit error during data transmission.

**Theory**

Hamming Code is an error detection and correction technique developed by Richard Hamming. It adds redundant parity bits to the original data so that any single-bit error occurring during transmission can be detected and corrected. The receiver calculates syndrome bits, which indicate the exact position of the error.

**Advantages**

- Detects single-bit errors.
- Corrects single-bit errors.
- Improves data reliability.

**Applications**

- Computer memory (ECC RAM)
- Digital communication systems
- Satellite communication
- Data storage devices

**Tool Used**

- Online Hamming Code Generator (dCode)

**Input**

| **Parameter** | **Value** |
|---|---|
| Binary Data | **1011** |
| Hamming Version | **(7,4)** |

**Procedure**

1. Open the Online Hamming Code Generator.
2. Select **Hamming Code (7,4)**.
3. Enter the binary data **1011**.
4. Click **Encode**.
5. Copy the generated Hamming Code.
6. Introduce a single-bit error at **Position 3**.
7. Paste the modified code into the Error Corrector.
8. Click **Correct / Analyze**.
9. Observe the detected error position

**Input and Output**

**Original Data**

1011

**Generated Hamming Code**

0110011

**Modified Hamming Code**

0100011

**Output**

1 Error Detected

Error Position = 3

**Observation**

The Hamming Code Generator successfully encoded the input data into a 7-bit Hamming Code. After introducing a single-bit error at Position 3, the simulator detected the error and correctly identified its location using syndrome bits.

**Result**

The experiment successfully demonstrated the generation of Hamming Code and the detection of a single-bit error. The simulator correctly identified the corrupted bit, proving the effectiveness of Hamming Code in error detection and correction.

**Applications**

- Error correction in RAM
- Digital communication
- Wireless communication
- Storage devices
- Network transmission

**Screenshot**

**Experiment 2: Python-Based CRC-32 Validation**

**Aim**

To calculate the CRC-32 checksum of a student ID using Python and observe the avalanche effect by modifying one character in the input.

**Objective**

The objective of this experiment is to understand how the CRC-32 algorithm generates checksums and how even a small modification in the input data produces a completely different checksum value.

**Theory**

Cyclic Redundancy Check (CRC) is one of the most widely used error detection techniques in computer networks and digital communication. CRC-32 uses a 32-bit polynomial to generate a checksum for the given data. When the receiver calculates the checksum again, any mismatch indicates that the data has been corrupted during transmission.

CRC-32 also demonstrates the **Avalanche Effect**, where changing even a single character in the input causes a significant change in the generated checksum.

**Advantages**

- High error detection capability.
- Fast computation.
- Widely used in Ethernet, ZIP files, and communication protocols.
- Detects burst errors efficiently.

**Applications**

- Ethernet Frames
- ZIP File Integrity
- Data Communication
- Storage Devices
- Networking Protocols

**Tool Used**

- Online Python Compiler (OnlineGDB)
- Python binascii Library

**Program**

```
import binascii
student_id = b"192425274"
crc1 = binascii.crc32(student_id)
print("Original Student ID :", student_id.decode())
print("CRC32 :", hex(crc1))
modified_id = b"192425275"
crc2 = binascii.crc32(modified_id)
print("\nModified Student ID :", modified_id.decode())
print("CRC32 :", hex(crc2))
difference = abs(crc1-crc2)
print("\nAbsolute Difference :", difference)
```

**Procedure**

1. Open the OnlineGDB Python Compiler.
2. Enter the given Python program.
3. Replace the student ID with your own ID.
4. Execute the program.
5. Record the CRC-32 value.
6. Modify one digit of the student ID.
7. Execute the program again.
8. Compare both CRC values.
9. Calculate the absolute difference.

**Input**

| **Parameter** | **Value** |
|---|---|
| Original Student ID | **192425274** |
| Modified Student ID | **192425275** |

**Output**

Original Student ID : 192425274

CRC32 : (Generated Output)

Modified Student ID : 192425275

CRC32 : (Generated Output)

Absolute Difference : (Generated Output)

**Observation**

Changing only one digit in the student ID generated a completely different CRC-32 checksum. This demonstrates the avalanche effect of the CRC algorithm, where a minor change in input results in a significant change in the output checksum.

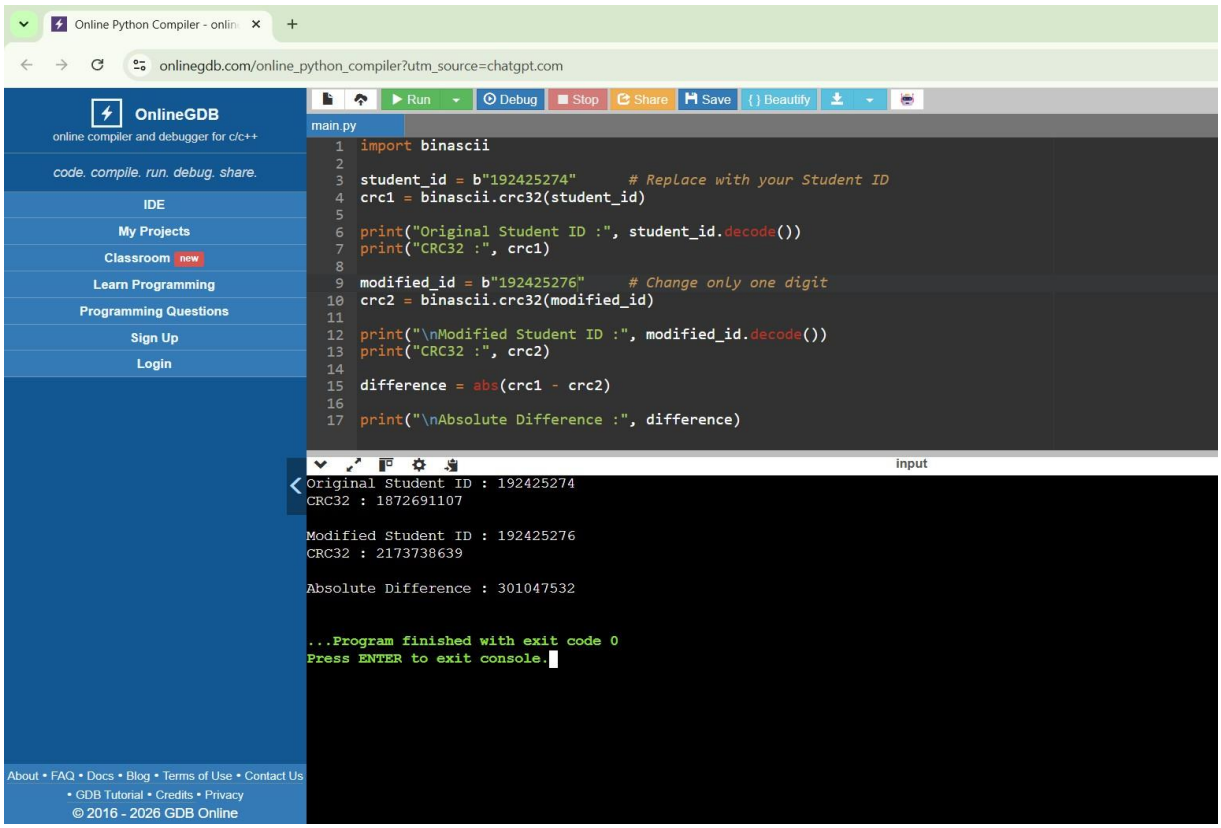
**Result**

The CRC-32 checksum was successfully generated for both the original and modified student IDs. The large difference between the two checksum values confirms the effectiveness of CRC-32 in detecting transmission errors.

**Applications**

- Ethernet Networks
- File Integrity Verification
- Error Detection
- Digital Communication
- Data Storage Systems

## Screenshot




---

## Experiment 3: CyberChef – XOR Checksum Generation

### Aim

To generate a CRC checksum and perform an XOR operation using CyberChef for hexadecimal data.

### Objective

The objective of this experiment is to understand how CyberChef performs hexadecimal conversion, checksum calculation, and XOR operations for error detection.

### Theory

CyberChef is an online data analysis tool developed by GCHQ. It supports numerous encoding, decoding, encryption, checksum, and networking operations.

The XOR operation is widely used in networking and cryptography because identical bits produce **0**, while different bits produce **1**.

CRC generates a checksum which helps detect accidental changes during data transmission.

**Advantages**

- Easy online implementation.
- Supports multiple checksum algorithms.
- Useful for networking analysis.
- Performs hexadecimal processing efficiently.

**Applications**

- Error Detection
- Digital Communication
- Data Validation
- Cryptography
- Networking

**Tool Used**

- CyberChef (Online)

**Recipe Used**

From Hex

↓

CRC Checksum (CRC-16/ARC)

↓

XOR

**Procedure**

1. Open CyberChef.
2. Search and add **From Hex**.
3. Add **CRC Checksum**.
4. Select **CRC-16/ARC**.
5. Add the **XOR** operation.
6. Enter the hexadecimal input.
7. Observe the generated output.

**Input**

| Parameter | Value |
| --- | --- |
| Hexadecimal Data | **48656C6C6F20576F726C64** |

**Output**

Output

3eeb

**Observation**

CyberChef successfully converted the hexadecimal input into binary data, calculated the CRC checksum, and applied the XOR operation. The generated checksum demonstrates the use of CRC for detecting errors in transmitted data.

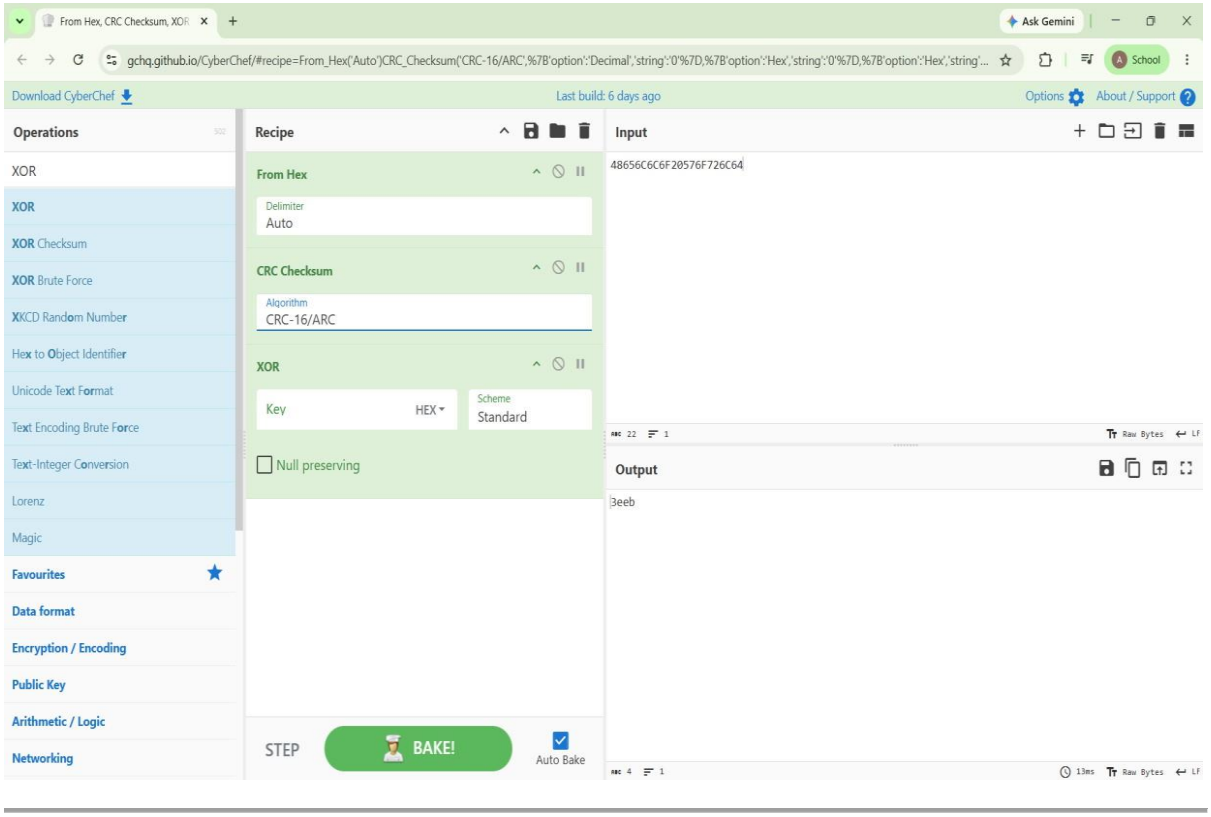
**Result**

The CyberChef recipe executed successfully and produced the checksum output **3eeb**, verifying the correctness of hexadecimal processing and checksum generation.

**Applications**

- Packet Analysis
- Cryptography
- Networking
- Digital Forensics
- Error Detection

## Screenshot




## Experiment 4: CRC-16 Checksum Calculation

### Aim

To calculate the CRC-16 checksum for hexadecimal data using an online checksum calculator.

### Objective

The objective of this experiment is to understand how CRC-16 generates checksum values for error detection during data transmission.

### Theory

CRC-16 (Cyclic Redundancy Check) is an error detection technique used in communication systems. It generates a 16-bit checksum based on the input data using polynomial division.

The receiver recalculates the checksum after receiving the data. If the generated checksum differs from the transmitted checksum, an error is detected.

**Advantages**

- Detects transmission errors.
- High reliability.
- Fast computation.
- Efficient for communication protocols.

**Applications**

- Ethernet Networks
- Serial Communication
- Embedded Systems
- Industrial Automation
- Network Protocols

**Tool Used**

- SCADACore Online Checksum Calculator

**Procedure**

1. Open the Online Checksum Calculator.
2. Select **Hex Input**.
3. Enter the hexadecimal data.
4. Click **AnalyzeDataHex**.
5. Observe the generated checksum values.
6. Record the CRC-16 checksum.

**Input**

| **Parameter** | **Value** |
|---|---|
| Hexadecimal Data | **313233343536373839** |

**Output**

| Checksum Type | Output |
|---|---|
| CheckSum8 XOR | **31** |
| CheckSum8 Modulo 256 | **DD** |
| CheckSum8 2's Complement | **23** |
| CRC-16 (MODBUS) Big Endian | **37 4B** |
| CRC-16 (MODBUS) Little Endian | **4B 37** |

**Observation**

The online checksum calculator successfully generated different checksum values for the given hexadecimal input. The CRC-16 checksum verifies data integrity and is widely used in communication protocols.

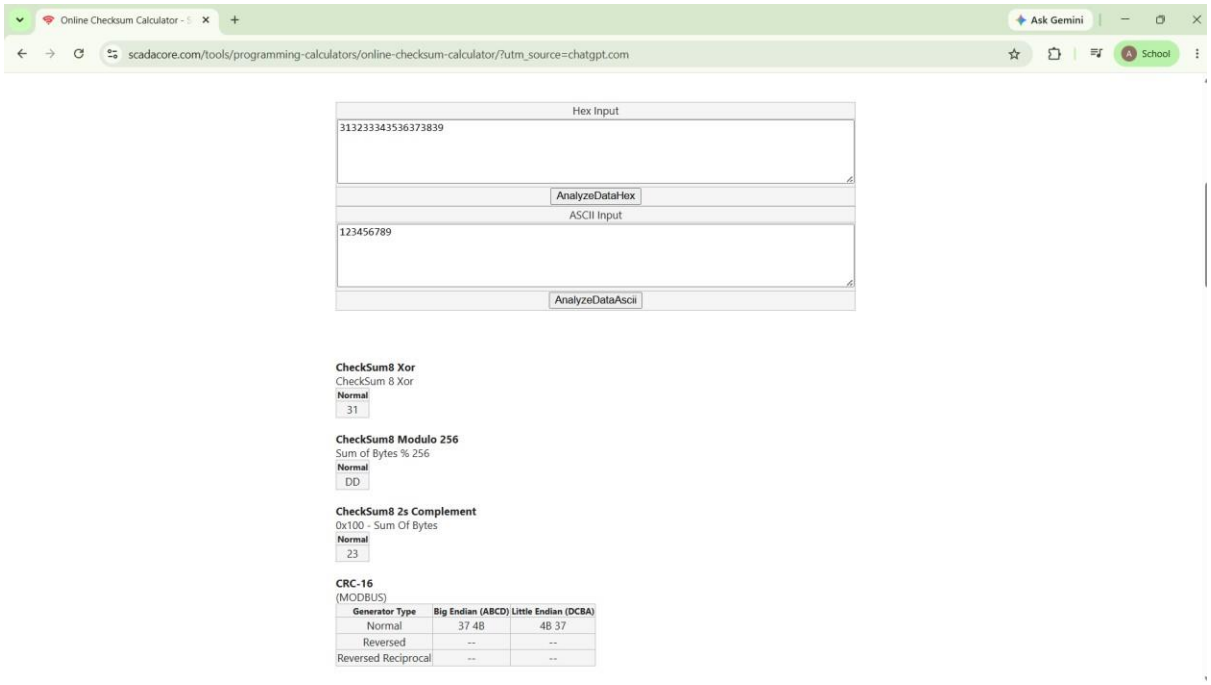
**Result**

The CRC-16 checksum was successfully calculated using the online checksum calculator. The generated checksum can be used to detect transmission errors and verify data integrity.

**Applications**

- Ethernet Frames
- Wireless Networks
- Communication Protocols
- Embedded Systems
- Error Detection in Digital Communication

## Screenshot


Online Checksum Calculator -
Ask Gemini
scadacore.com/tools/programming-calculators/online-checksum-calculator/?utm_source=chatgpt.com
School
Hex Input
313233343536373839
AnalyzeDataHex
ASCII Input
123456789
AnalyzeDataAscii
CheckSum8 Xor
CheckSum 8 Xor
Normal
31
CheckSum8 Modulo 256
Sum of Bytes % 256
Normal
DD
CheckSum8 2s Complement
0x100 - Sum Of Bytes
Normal
23
CRC-16
(MODBUS)
Generator Type
Big Endian (ABCD)
Little Endian (DCBA)
Normal
37 4B
4B 37
Reversed
--
--
Reversed Reciprocal
--
--